Project Libre Document

CampusSphere

By

Rajwardhan Singh (24CSB1A52)

Ramitha Ramesh (24CSB1A53)

Rayi Charan Harsha (24CSB1A54)

Part 1: Work Breakdown Structure (WBS)

1.1 Overview

The CampusSphere project WBS is structured into nine major phases, each with detailed sub-tasks and associated costs. The hierarchy demonstrates clear project organization and scope definition, extending the standard SDLC phases with a dedicated Recommendation Engine Development phase for the continuously evolving, survey-seeded recommendation system.

1.2 Major Project Phases

1) Feasibility Study

a) Define Project Scope & Identify Stakeholders

b) Requirement Gathering

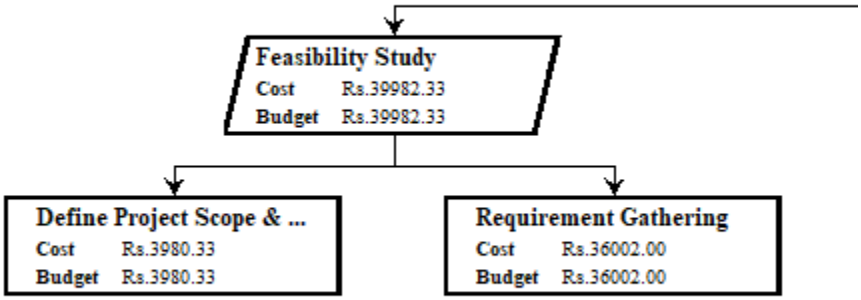


Fig1.1 Feasibility Study Breakdown

2) Requirement Analysis

Purpose: Collect detailed functional and non-functional requirements and create a Software Requirements Specification (SRS) document.

a) Assumptions & Dependencies

b) Approval & Documentation Formatting

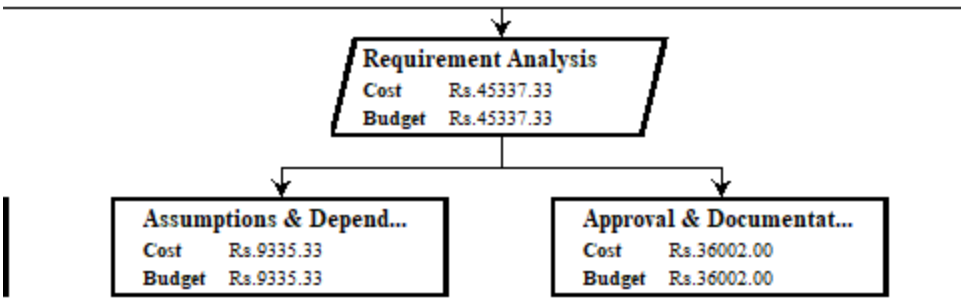


Fig1.2 Requirement Analysis Breakdown

3) System Design

a) System Architecture & DFD

b) API Specification

c) Design Data Models / DB Schema Design

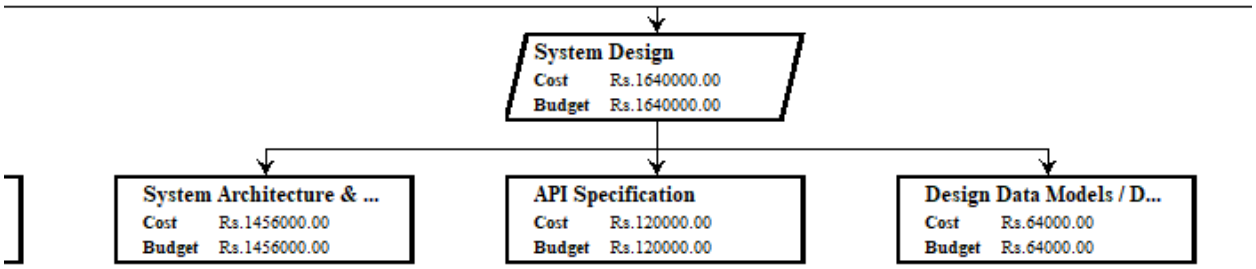


Fig1.3 System Design Breakdown

4) Database Development

a) Database Schema Creation (Users, Roles, Skills, UserSkills, Organizations, Opportunities, Categories, Applications, Bookmarks, Notifications, Achievements, Resumes, Recommendations, Comments, Tags, OpportunityTags)

b) Data Backup & Migration Setup

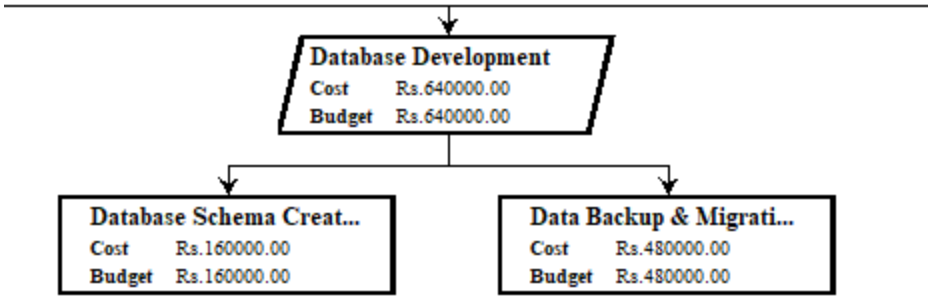


Fig1.4 Database Development Breakdown

5) UI/UX Development

a) Wireframes & Mockups

b) Implement Front-End Code

c) UI Integration with Back-End

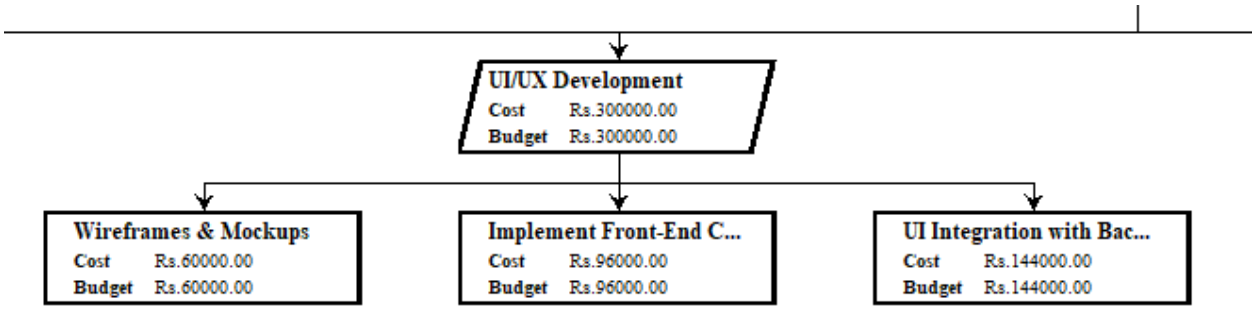


Fig1.5 UI/UX Development Breakdown

6) Recommendation Engine Development

Purpose: Build the evolving recommendation system: an initial-survey-seeded skill/interest profile that is continuously refined as students apply, bookmark, and interact with opportunities.

a) Initial Survey Module

b) Recommendation Algorithm Development

c) Continuous Feedback Loop Integration

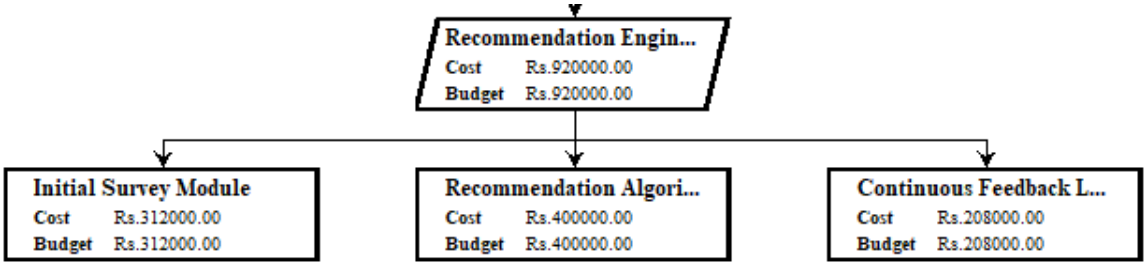


Fig1.6 Recommendation Engine Development Breakdown

7) Testing

a) Unit Testing

b) Integration Testing

c) UAT & Bug Fixing

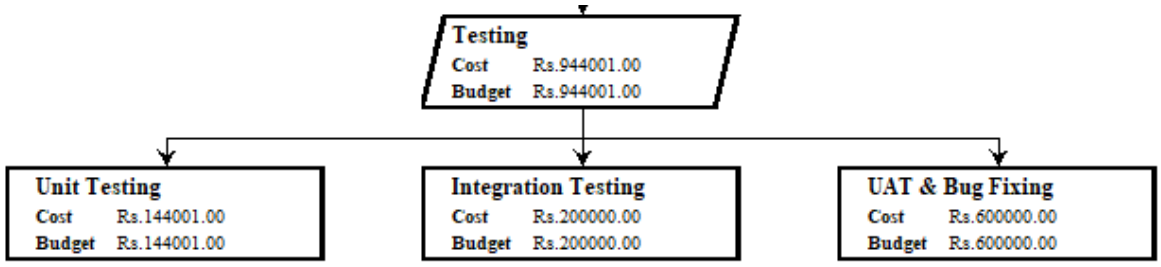


Fig1.7 Testing Breakdown

8) Deployment

a) Prepare Production Environment

b) Deploy Software & Testing

c) Set-up Monitoring & Logging

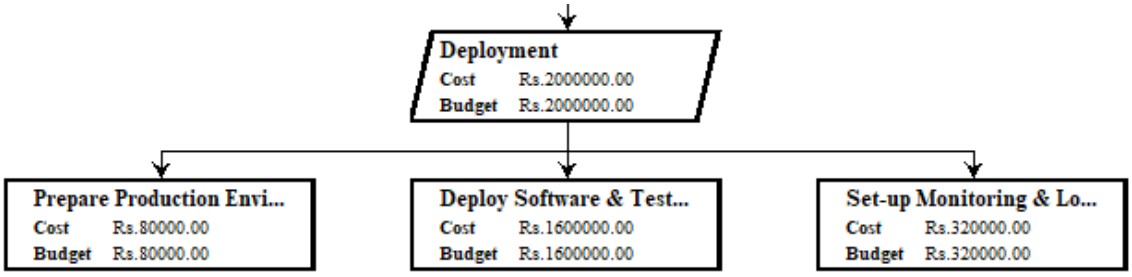


Fig1.8 Deployment Breakdown

9) Maintenance & Enhancement

a) Monitor System Performance

b) Recommendation Tuning & Notification Setup

c) Gather Feedback

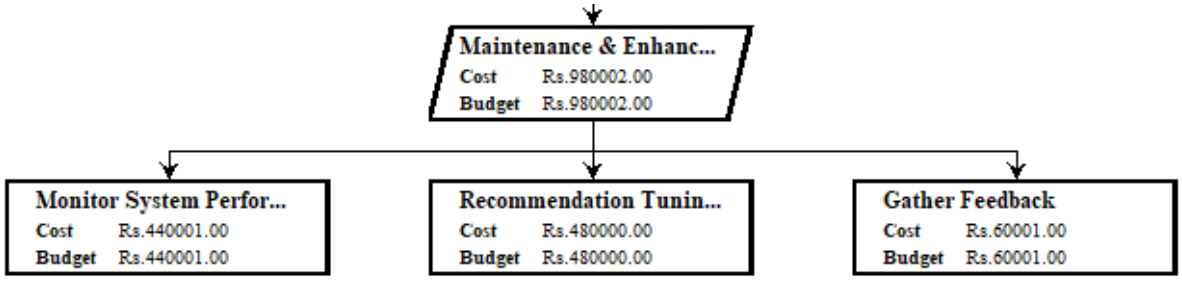
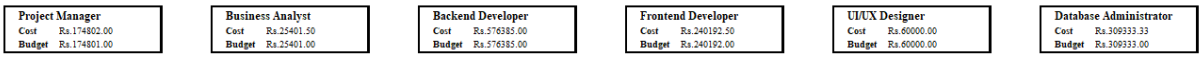


Fig1.9 Maintenance & Enhancement Breakdown

Resource Allocation

The table below summarizes “Who Does What” — the tasks each resource is assigned to, total work (hours), and the resulting cost.



Part 2: Developing a Project Plan

CampusSphere involves a range of functionalities — opportunity posting and discovery, role-based access, applications and bookmarking, notifications, and an evolving recommendation engine — which are broken down into tasks and sub-tasks to make the development process manageable in ProjectLibre.

Activities, Durations, and Dependencies

	Name	Duration	Start	Finish	Predecessors	Resource Names
1	**CampusSphere – Campu**	54 days	**20/8/26 8:00 AM**	**3/11/26 5:00 PM**		
2	**Feasibility Study**	3 days	**20/8/26 8:00 AM**	**24/8/26 5:00 PM**		
3	Define Project Scope & Ic	1 day	20/8/26 8:00 AM	20/8/26 5:00 PM		Project Manager
4	Requirement Gathering	2 days	21/8/26 8:00 AM	24/8/26 5:00 PM	3	Project Manager;Business A...
5	**Requirement Analysis**	3 days	**25/8/26 8:00 AM**	**27/8/26 5:00 PM**		
6	Assumptions & Depender	1 day	25/8/26 8:00 AM	25/8/26 5:00 PM	4	Business Analyst
7	Approval & Documentatic	2 days	26/8/26 8:00 AM	27/8/26 5:00 PM	6	Business Analyst;Project Ma...
8	**System Design**	5 days	**28/8/26 8:00 AM**	**3/9/26 5:00 PM**		
9	System Architecture & DI	2 days	28/8/26 8:00 AM	31/8/26 5:00 PM	7	Backend Developer;DevOps...
10	API Specification	2 days	1/9/26 8:00 AM	2/9/26 5:00 PM	9	Backend Developer
11	Design Data Models / DB	1 day	3/9/26 8:00 AM	3/9/26 5:00 PM	10	Database Administrator
12	**Database Developmen**	5 days	**4/9/26 8:00 AM**	**10/9/26 5:00 PM**		
13	Database Schema Creati	3 days	4/9/26 8:00 AM	8/9/26 5:00 PM	11	Database Administrator
14	Data Backup & Migration	2 days	9/9/26 8:00 AM	10/9/26 5:00 PM	13	Database Administrator;De...
15	**UI/UX Development**	7 days	**4/9/26 8:00 AM**	**14/9/26 5:00 PM**		
16	Wireframes & Mockups	3 days	4/9/26 8:00 AM	8/9/26 5:00 PM	11	UI/UX Designer
17	Implement Front-End Coc	2 days	9/9/26 8:00 AM	10/9/26 5:00 PM	16	Frontend Developer
18	UI Integration with Back-	2 days	11/9/26 8:00 AM	14/9/26 5:00 PM	17	Frontend Developer;Backen...
19	**Recommendation Engi**	7 days	**11/9/26 8:00 AM**	**21/9/26 5:00 PM**		
20	Initial Survey Module	2 days	11/9/26 8:00 AM	14/9/26 5:00 PM	14	AI/ML Engineer;Frontend D...
21	Recommendation Algorith	4 days	15/9/26 8:00 AM	18/9/26 5:00 PM	20	AI/ML Engineer
22	Continuous Feedback Loc	1 day	21/9/26 8:00 AM	21/9/26 5:00 PM	21	AI/ML Engineer;Backend De...
23	**Testing**	9 days	**22/9/26 8:00 AM**	**2/10/26 5:00 PM**		
24	Unit Testing	2 days	22/9/26 8:00 AM	23/9/26 5:00 PM	14;18;22	QA / Test Engineer;Project ...
25	Integration Testing	2 days	24/9/26 8:00 AM	25/9/26 5:00 PM	24	QA / Test Engineer
26	UAT & Bug Fixing	5 days	28/9/26 8:00 AM	2/10/26 5:00 PM	25	QA / Test Engineer
27	**Deployment**	13 days	**5/10/26 8:00 AM**	**21/10/26 5:00 PM**		

	Name	Duration	Start	Finish	Predecessors	Resource Names
8	**System Design**	5 days	**28/8/26 8:00 AM**	**3/9/26 5:00 PM**		
9	System Architecture & DI	2 days	28/8/26 8:00 AM	31/8/26 5:00 PM	7	Backend Developer;DevOps...
10	API Specification	2 days	1/9/26 8:00 AM	2/9/26 5:00 PM	9	Backend Developer
11	Design Data Models / DB	1 day	3/9/26 8:00 AM	3/9/26 5:00 PM	10	Database Administrator
12	**Database Developmen**	5 days	**4/9/26 8:00 AM**	**10/9/26 5:00 PM**		
13	Database Schema Creati	3 days	4/9/26 8:00 AM	8/9/26 5:00 PM	11	Database Administrator
14	Data Backup & Migration	2 days	9/9/26 8:00 AM	10/9/26 5:00 PM	13	Database Administrator;De...
15	**UI/UX Development**	7 days	**4/9/26 8:00 AM**	**14/9/26 5:00 PM**		
16	Wireframes & Mockups	3 days	4/9/26 8:00 AM	8/9/26 5:00 PM	11	UI/UX Designer
17	Implement Front-End Coc	2 days	9/9/26 8:00 AM	10/9/26 5:00 PM	16	Frontend Developer
18	UI Integration with Back-	2 days	11/9/26 8:00 AM	14/9/26 5:00 PM	17	Frontend Developer;Backen...
19	**Recommendation Engi**	7 days	**11/9/26 8:00 AM**	**21/9/26 5:00 PM**		
20	Initial Survey Module	2 days	11/9/26 8:00 AM	14/9/26 5:00 PM	14	AI/ML Engineer;Frontend D...
21	Recommendation Algorith	4 days	15/9/26 8:00 AM	18/9/26 5:00 PM	20	AI/ML Engineer
22	Continuous Feedback Loc	1 day	21/9/26 8:00 AM	21/9/26 5:00 PM	21	AI/ML Engineer;Backend De...
23	**Testing**	9 days	**22/9/26 8:00 AM**	**2/10/26 5:00 PM**		
24	Unit Testing	2 days	22/9/26 8:00 AM	23/9/26 5:00 PM	14;18;22	QA / Test Engineer;Project ...
25	Integration Testing	2 days	24/9/26 8:00 AM	25/9/26 5:00 PM	24	QA / Test Engineer
26	UAT & Bug Fixing	5 days	28/9/26 8:00 AM	2/10/26 5:00 PM	25	QA / Test Engineer
27	**Deployment**	13 days	**5/10/26 8:00 AM**	**21/10/26 5:00 PM**		
28	Prepare Production Envir	1 day	5/10/26 8:00 AM	5/10/26 5:00 PM	26	DevOps Engineer
29	Deploy Software & Testir	10 days	6/10/26 8:00 AM	19/10/26 5:00 PM	28	DevOps Engineer
30	Set-up Monitoring & Logg	2 days	20/10/26 8:00 AM	21/10/26 5:00 PM	29	DevOps Engineer
31	**Maintenance & Enhanc**	9 days	**22/10/26 8:00 AM**	**3/11/26 5:00 PM**		
32	Monitor System Performa	2 days	22/10/26 8:00 AM	23/10/26 5:00 PM	30	DevOps Engineer;Project M...
33	Recommendation Tuning	3 days	26/10/26 8:00 AM	28/10/26 5:00 PM	32	AI/ML Engineer;Support Staff
34	Gather Feedback	4 days	29/10/26 8:00 AM	3/11/26 5:00 PM	33	Support Staff;Project Mana...

Network Diagram

The network diagram outlines the phase-level flow of the CampusSphere development project. It begins with Feasibility Study and Requirement Analysis, proceeds through System Design, then fans out into three parallel implementation tracks — Database Development, UI/UX Development, and

Recommendation Engine Development — which converge again at Testing, followed by Deployment and Maintenance & Enhancement.

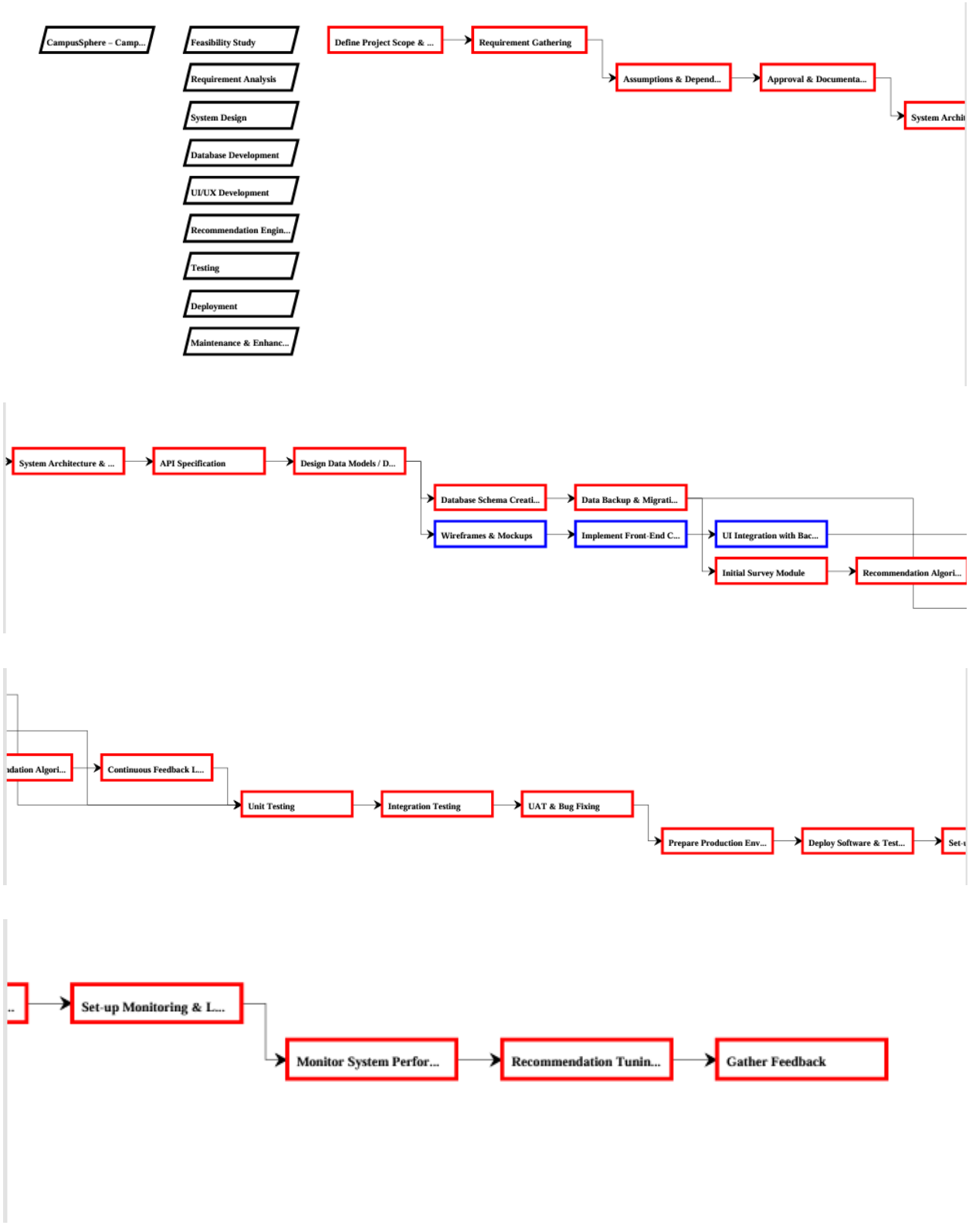


Fig 2.1 Network / Phase Flow Diagram

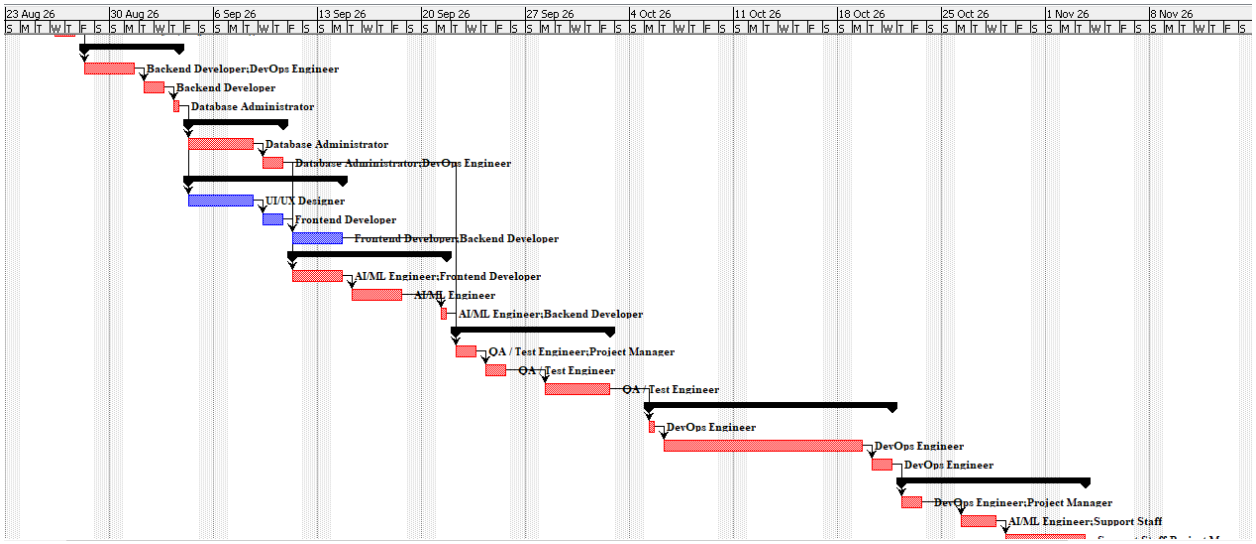
Critical Path and Duration

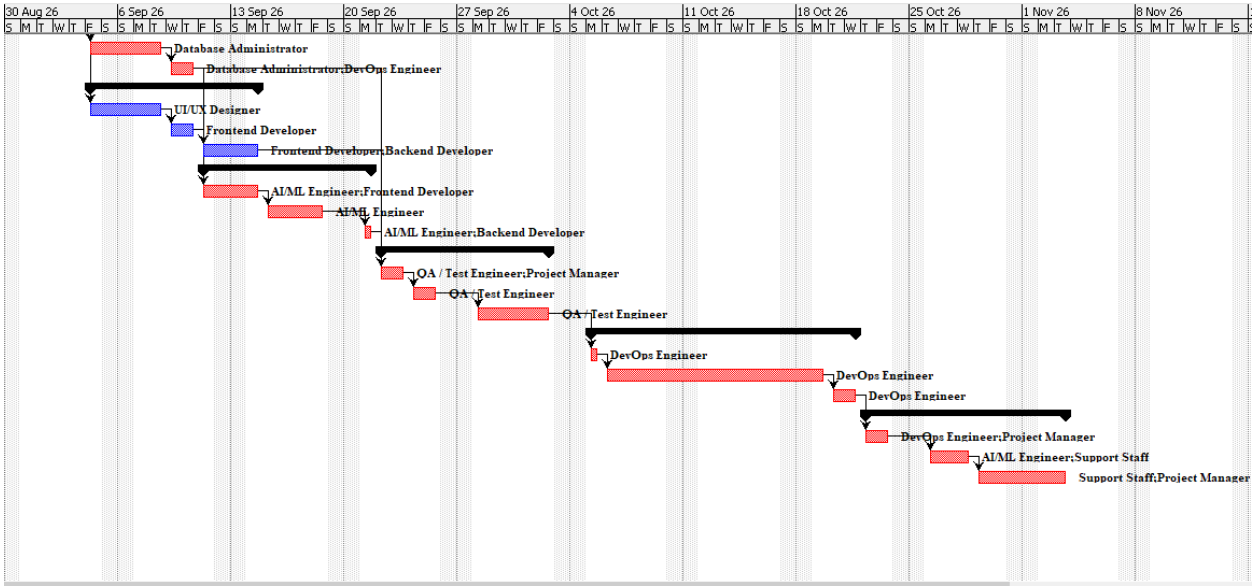
The critical path is the longest sequence of dependent tasks in the project; tasks on this path have zero slack, so any delay to them delays the entire project. For CampusSphere, the critical path runs through Requirement Gathering → Requirement Analysis → System Design → Database Development → Recommendation Engine Development → Testing → Deployment → Maintenance & Enhancement, since the Recommendation Engine cannot begin its survey-driven matching logic until the database schema (skills, opportunities, applications) is available.

Total Project Duration: 53 working days (approx. 1.8 months), from 20/08/26 to 03/11/26.

Gantt Chart

A Gantt chart visually represents each task, its duration, and its dependencies across the timeline. In ProjectLibre, open the imported CampusSphere.xml file and use View → Gantt Chart; right-click a task and choose “Critical Path” styling to highlight critical tasks in red, as shown below (generated from the computed schedule).





Part 3: Scheduling Resources and Costs

1. Resource Allocation

A resource histogram shows how a resource's utilization varies over the project timeline, which helps identify overallocation. Below is an illustrative histogram for the DevOps Engineer, generated from the schedule.

2. Types of Resources

Human Resources

Project Manager: Manages scope, gathers requirements, and coordinates stakeholders across student, faculty, recruiter, and admin roles.

Business Analyst: Documents requirements, writes the SRS, and formats approvals and documentation.

Backend Developer: Implements the FastAPI services, authentication (JWT), and API layer.

Frontend Developer: Implements the React.js / Tailwind CSS front end and connects it to the back end.

UI/UX Designer: Creates wireframes and mockups and ensures a consistent, accessible design system.

Database Administrator (DBA): Designs and maintains the PostgreSQL schema, backups, and migrations.

AI/ML Engineer: Builds the initial-survey-seeded recommendation engine and its continuous feedback loop.

QA / Test Engineer: Performs unit, integration, and UAT testing and coordinates bug fixing.

DevOps Engineer: Handles containerized deployment (Docker), production environments, and monitoring/logging.

Support Staff: Assists with feedback gathering and ongoing documentation.

Software Resources

VSCode: Code editor for development.

Figma: Design tool for wireframes and mockups.

Postman: API testing and documentation.

GitHub: Version control and collaboration.

Hardware Resources

Web Server: Hosts the React front end and static assets.

Database Server: Hosts the PostgreSQL database.

Backup Server: Provides data backup and disaster recovery.

Cloud Infrastructure (AWS): Scalable hosting and deployment (Docker on Render/Railway/AWS).

RBS for Human Resources

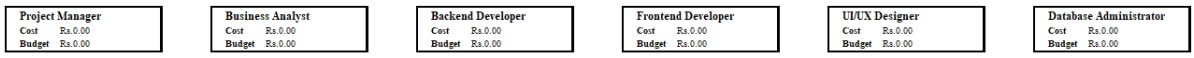


Fig 3 RBS

Part 4: Reducing Project Duration

1. Schedule Compression Techniques

Schedule compression uses fast tracking (running tasks concurrently) and crashing (adding resources to critical tasks) to shorten the project duration without changing scope.

Fast Tracking

Analysis of the Deployment and Maintenance & Enhancement phases showed that “Set-up Monitoring & Logging” and the start of “Monitor System Performance” could be overlapped, since monitoring dashboards can be configured while the first performance readings are being collected. Similarly, “Recommendation Tuning & Notification Setup” can begin as soon as initial performance data is available rather than waiting for the full monitoring window to close. Converting these Finish-to-Start links to Start-to-Start with a lag reduces the Maintenance & Enhancement phase from 18.5 days to an estimated 11–12 days.

Crashing

On the critical path, “UAT & Bug Fixing” (15 days) and “Deploy Software & Testing” (20 days) are the largest single contributors to project duration. Adding a second QA/Test Engineer to UAT & Bug Fixing and a second DevOps Engineer to the deployment task would allow both to be crashed to roughly 60% of their original duration (9 days and 12 days respectively), at the cost of the additional resources' standard rate for the compressed period. This trades a modest cost increase for a meaningful reduction in overall project duration, which is worthwhile if the CampusSphere pilot needs to launch before the start of a new academic semester.

Benefits Achieved

- Reduced project duration without compromising major milestones or deliverables.
- Cost efficiency by applying fast tracking first, reserving crashing for the highest-impact critical-path tasks.
- Improved flexibility to adapt to the academic calendar or changing stakeholder requirements.

Steps to Reproduce in ProjectLibre

1. Create the Summary Task (CampusSphere – Campus Opportunity Hub) and the key phase tasks.
2. Add key task dependencies (Finish-to-Start links as listed in Part 2).
3. Insert sub-tasks under each phase and indent them.
4. Add sub-task dependencies within each phase.
5. Estimate duration for every task (durations provided in Part 2's task table).
6. Change the Working Calendar to a standard Mon–Fri, 8:00–12:00 / 13:00–17:00 week (already set in the imported XML).
7. Save a baseline once the schedule looks correct (Project → Set Baseline).
8. Insert extra columns — Deadline, Total Slack, Cost, Work — via right-click on the column header → Insert Column.
9. View Total Slack in the Gantt chart to confirm the critical path (zero-slack tasks).

10. Update Constraint Type on tasks where needed (e.g. “Must Start On” for a fixed pilot-launch date) and verify on the Gantt chart.

11. Update Percent Complete as work progresses and verify the change on the Gantt chart.

12. Create the Resources listed in Part 3 and set their Group and Standard/Overtime Rate.

13. Build a resource hierarchy (e.g. DevOps Engineers and QA Testers reporting to the Project Manager).

14. Add Materials (VSCode, Figma, Postman, cloud/hardware) and set a cost per use where applicable.

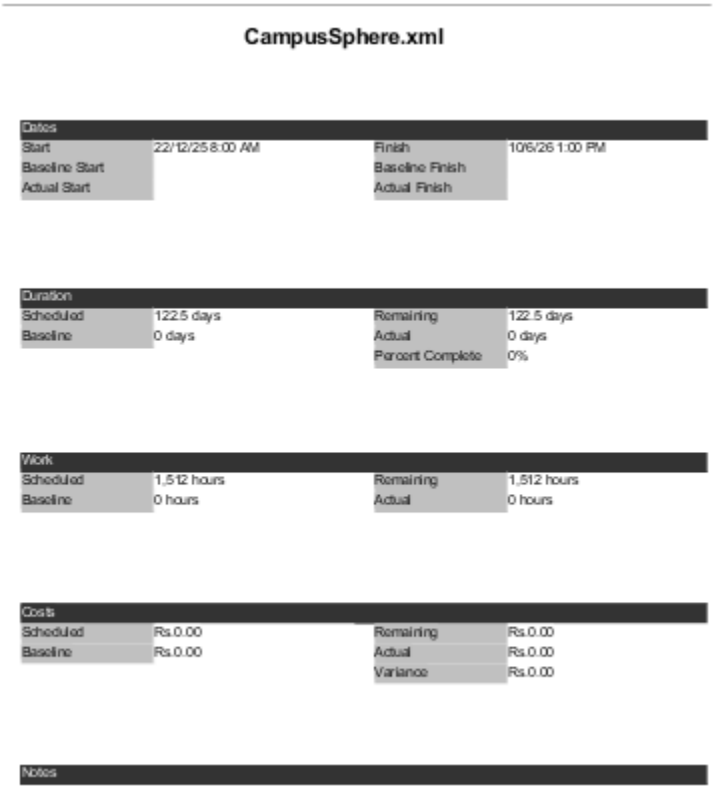
15. Assign resources to tasks and verify assignments on the Gantt chart.

16. View the resource histogram (View → Resources → Resource Usage) and check for overallocation (>100%).

17. Resolve any overallocation by leveling resources or adjusting assignment units.

18. Explore the WBS, RBS, and Network Diagram views under the View tab.

19. Save the project as CampusSphere.pod (or re-export to XML) once complete.



CampusSphere.xml

Dates			
Start	22/12/25 8:00 AM	Finish	10/6/26 1:00 PM
Baseline Start		Baseline Finish	
Actual Start		Actual Finish	

Duration			
Scheduled	122.5 days	Remaining	122.5 days
Baseline	0 days	Actual	0 days
		Percent Complete	0%

Work			
Scheduled	1,512 hours	Remaining	1,512 hours
Baseline	0 hours	Actual	0 hours

Costs			
Scheduled	Rs.0.00	Remaining	Rs.0.00
Baseline	Rs.0.00	Actual	Rs.0.00
		Variance	Rs.0.00

Notes